PRACTICE SET – 3

Insert 5 members (make sure one member has no loans) and 8 loans before starting.

```sql
CREATE DATABASE library ;
USE library ;

CREATE TABLE members (
member_id INT PRIMARY KEY ,
mname VARCHAR (50) ,
city VARCHAR (30)
);

CREATE TABLE loans (
loan_id INT PRIMARY KEY ,
member_id INT ,
book_title VARCHAR (80) ,
fine INT ,
FOREIGN KEY ( member_id ) REFERENCES members ( member_id )
);

INSERT INTO members
VALUES
(101,"Michael","Kathmandu"),
(102,"Jackson","Pokhara"),
(103,"Jordan","Hetauda"),
(104,"Tyson","Biratnagar"),
(105,"Terry","Birgunj");
```

```sql
INSERT INTO loans
VALUES
(1,101,"Harry Potter",500),
(2,101,"Muna Madan",200),
(3,102,"Gau Khanne Katha",200),
(4,102,"Muna Madan",400),
(5,103,"Spider-Man",600),
(6,103,"Himalayas",200),
(7,104,"Khullam Khulla",200),
(8,104,"Muna Madan",200);
```

DATABASE OUTPUT:

Result Grid | Filter Rows:

member_id	mname	city
101	Michael	Kathmandu
102	Jackson	Pokhara
103	Jordan	Hetauda
104	Tyson	Biratnagar
105	Terry	Birgunj
NULL	NULL	NULL

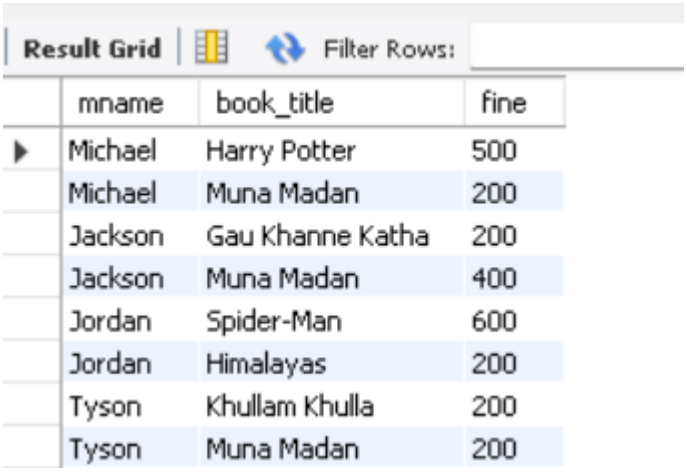
Result Grid | Filter Rows: | Edit: | Export/

loan_id	member_id	book_title	fine
1	101	Harry Potter	500
2	101	Muna Madan	200
3	102	Gau Khanne Katha	200
4	102	Muna Madan	400
5	103	Spider-Man	600
6	103	Himalayas	200
7	104	Khullam Khulla	200
8	104	Muna Madan	200

1. **INNER JOIN members and loans. Predict the row count before running.**

```
SELECT members.mname , loans.book_title,loans.fine
FROM members
INNER JOIN loans
ON members.member_id = loans.member_id;
```

DATABASE OUTPUT:



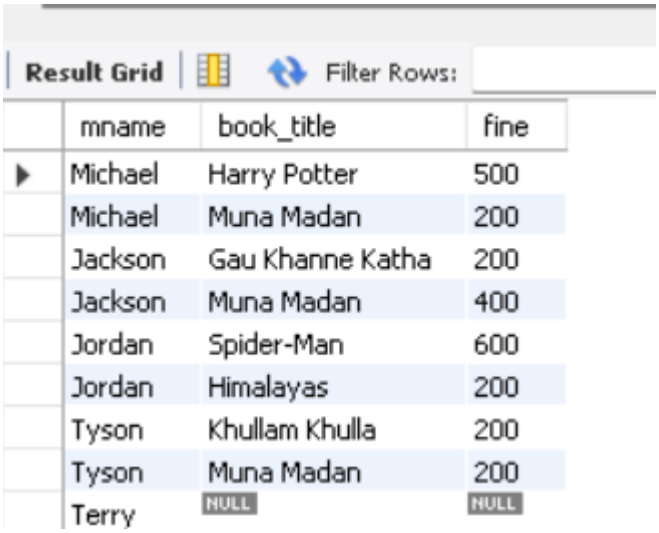
Result Grid | Filter Rows:

mname	book_title	fine
Michael	Harry Potter	500
Michael	Muna Madan	200
Jackson	Gau Khanne Katha	200
Jackson	Muna Madan	400
Jordan	Spider-Man	600
Jordan	Himalayas	200
Tyson	Khullam Khulla	200
Tyson	Muna Madan	200

1. **LEFT JOIN showing all members with their books. Who shows NULL and why?**

```
47
48 •    SELECT members.mname , loans.book_title,loans.fine
49      FROM members
50      LEFT JOIN loans
51      ON members.member_id = loans.member_id ;
52
```

DATABASE OUTPUT:



Result Grid | Filter Rows:

mname	book_title	fine
Michael	Harry Potter	500
Michael	Muna Madan	200
Jackson	Gau Khanne Katha	200
Jackson	Muna Madan	400
Jordan	Spider-Man	600
Jordan	Himalayas	200
Tyson	Khullam Khulla	200
Tyson	Muna Madan	200
Terry	NULL	NULL

3. Swap the two tables in the LEFT JOIN. Which earlier join does it now behave like?

```
53
54 •    SELECT members.mname , loans.book_title
55      FROM loans
56      LEFT JOIN members
57      ON loans.member_id = members.member_id
58
59
```

DATABASE OUTPUT:

Result Grid | Filter Rows:

mname	book_title
Michael	Harry Potter
Michael	Muna Madan
Jackson	Gau Khanne Katha
Jackson	Muna Madan
Jordan	Spider-Man
Jordan	Himalayas
Tyson	Khullam Khulla
Tyson	Muna Madan

4. RIGHT JOIN from members. What appears, and is any row NULL? Why not?

```
61  SELECT members.mname , loans.book_title, loans.fine
62  FROM loans
63  RIGHT JOIN members
64  ON members.member_id = loans.member_id ;
65
```

DATABASE OUTPUT:

Result Grid | Filter Rows:

mname	book_title	fine
Michael	Harry Potter	500
Michael	Muna Madan	200
Jackson	Gau Khanne Katha	200
Jackson	Muna Madan	400
Jordan	Spider-Man	600
Jordan	Himalayas	200
Tyson	Khullam Khulla	200
Tyson	Muna Madan	200
Terry	NULL	NULL

5. Write the FULL JOIN using UNION. How many rows?

```
56
57 •  SELECT members.member_id, members.mname , loans.book_title,loans.fine
58    FROM members
59    LEFT JOIN loans
70    ON members.member_id = loans.member_id
71
72    UNION
73
74    SELECT members.member_id, members.mname , loans.book_title,loans.fine
75    FROM members
76    RIGHT JOIN loans
77    ON members.member_id = loans.member_id;
78
```

DATABASE OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	mname	book_title	fine
101	Michael	Harry Potter	500
101	Michael	Muna Madan	200
102	Jackson	Gau Khanne Katha	200
102	Jackson	Muna Madan	400
103	Jordan	Spider-Man	600
103	Jordan	Himalayas	200
104	Tyson	Khullam Khulla	200
104	Tyson	Muna Madan	200
105	Terry	NULL	NULL

Result 13

6. Exclusive join: members who borrowed nothing. Which IS NULL check?

```
78
79 •  SELECT members.member_id, members.mname, members.city
80    FROM members
81    LEFT JOIN loans
82    ON members.member_id = loans.member_id
83    WHERE loans.loan_id IS NULL ;
84
```

DATABASE OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	mname	city
105	Terry	Birgunj

7. Show member names with book titles only for fines above 500 (join + WHERE).

```
84
85 •  SELECT members.mname , loans.book_title , loans.fine
86    FROM members
87    INNER JOIN loans
88    ON members.member_id = loans.member_id
89    WHERE loans.fine > 500 ;
```

DATABASE OUTPUT:

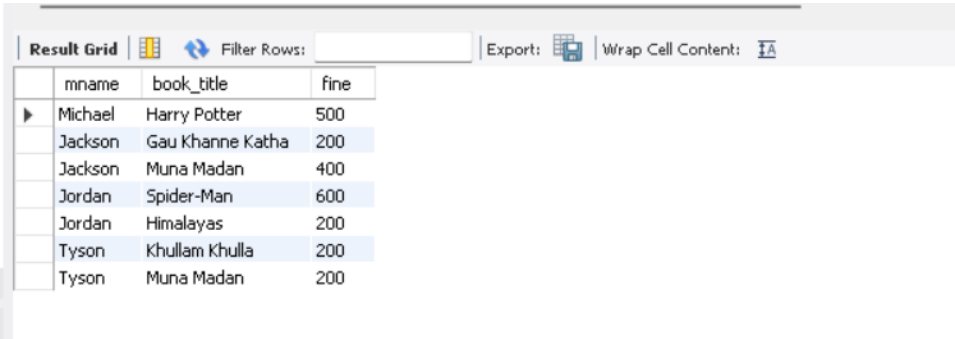
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

mname	book_title	fine
Jordan	Spider-Man	600

8. Delete one loan, run the INNER JOIN again. Which rows disappeared?

```
90
91 •    DELETE FROM loans WHERE loan_id = 2 ;
```

DATABASE OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

mname	book_title	fine
Michael	Harry Potter	500
Jackson	Gau Khanne Katha	200
Jackson	Muna Madan	400
Jordan	Spider-Man	600
Jordan	Himalayas	200
Tyson	Khullam Khulla	200
Tyson	Muna Madan	200

9. In your own words: when do you choose LEFT over INNER? Give a library example.

```
We choose a LEFT JOIN over an INNER JOIN when we want to keep all reco
in the secondary table
```

10. Bonus self join: add a referred by column to members (who invited whom) and list inviter with invited.

```
98
99  UPDATE members
100 SET referred_by = 101
101 WHERE member_id IN (102, 103);
102
103 UPDATE members
104 SET referred_by = 102
105 WHERE member_id = 104;
106
107
108 SELECT inviter.mname AS inviter, invited.mname AS invited
109 FROM members AS invited
110 INNER JOIN members AS inviter
111 ON invited.referred_by = inviter.member_id;
```

DATABASE OUTPUT:

Result Grid | Filter Rows:

inviter	invited
Michael	Jackson
Michael	Jordan
Jackson	Tyson